Certificate

This is to certify that the project report titled *"Design and Implementation of a Pipelined RV32I Processor with Branch Prediction and Custom ML Instructions"* submitted by **Vishnu Dheepak** is a bona fide record of work carried out under my supervision.

_____

Supervisor

Declaration

I hereby declare that the work presented in this report titled *"Design and Implementation of a Pipelined RV32I Processor with Branch Prediction and Custom ML Instructions"* is my own and has not been submitted elsewhere for the award of any other degree or diploma. All sources used have been duly acknowledged.

_____

Vishnu Dheepak

Acknowledgements

I would like to thank ...

Abstract

This report presents the design, implementation, and analysis of a 5-stage pipelined RV32I processor in Verilog, targeted at the Xilinx Zynq Z-7020 (Zybo Z2) FPGA. The processor implements the full RV32I base integer instruction set, the M-extension `MUL` instruction, and three custom machine-learning–oriented instructions on the RISC-V `custom-0` opcode: a three-operand multiply–accumulate (`MAC`), a rectified linear unit (`RELU`), and a cycle-counter read (`RDCYC`). The design is verified across three configurations (single-cycle, pipeline without branch prediction, pipeline with a 16-entry BTB and 2-bit saturating predictor) on a suite of nine benchmarks. Functional correctness is established by an automated 27-run validation harness; performance is reported in terms of instructions retired, CPI, and post-synthesis-derived real execution time. The custom MAC instruction achieves up to 3.45× speedup over the software-multiply baseline. Synthesis-based timing analysis reveals that the single-cycle combinational multiplier dominates the pipeline's critical path, motivating a multi-stage pipelined multiplier as future work.

Contents

List of Figures

List of Tables

Chapter 1

Introduction

1.1 Motivation

1.2 Problem Statement

1.3 Objectives

- Design a 5-stage pipelined RV32I-like processor in Verilog.
- Implement RAW-hazard handling via data forwarding plus stalls.
- Improve performance with a BTB-based dynamic branch predictor.
- Extend the ISA with custom ML-oriented instructions (`MAC`, `RELU`, `RDCYC`).
- Verify functionally and analyse performance against single-cycle and pipeline-without-BTB baselines.
- Synthesise on Zynq Z-7020 and analyse post-synthesis timing.

1.4 Scope and Contributions

1.5 Report Organisation

Chapter 2

Background

2.1 The RISC-V ISA

2.2 Pipelined Processor Fundamentals

2.3 Hazards

2.3.1 Structural Hazards

2.3.2 Data Hazards (RAW) and Forwarding

2.3.3 Control Hazards and Branch Penalty

2.4 Branch Prediction

2.5 FPGA Implementation Considerations

2.6 Related Work

Chapter 3

Instruction Set Architecture

3.1 Design Principles

3.2 Register File Specification

3.3 Instruction Formats

3.3.1 R-type

3.3.2 I-type

3.3.3 S-type

3.3.4 B-type

3.3.5 U-type

3.3.6 J-type

3.4 Bit-field Allocation

3.5 Implemented RV32I Base Instructions

3.6 M-Extension Instruction

3.7 Custom-0 Instructions

3.7.1 MAC — Three-Operand Multiply-Accumulate

3.7.2 RELU — Rectified Linear Unit

3.7.3 RDCYC — Cycle Counter Read

3.8 Encoding Summary

Chapter 4

Processor Design and Implementation

This chapter consolidates the architectural design and Verilog implementation across all phases. Each subsystem is described in terms of its function, interface, and design decisions; cross-cutting verification status is summarised at the end.

4.1 Top-Level Block Diagram

4.2 Instruction Fetch (IF) Stage

4.2.1 Program Counter

4.2.2 Instruction Memory

4.2.3 Branch Target Buffer

4.3 Instruction Decode (ID) Stage

4.3.1 Control Unit

4.3.2 Immediate Generator

4.3.3 Three-Port Register File

4.3.4 WB→ID Bypass

4.4 Execute (EX) Stage

4.4.1 ALU and ALU Control

4.4.2 Branch Unit

4.4.3 Multiplier and MAC Unit

4.4.4 EX-Stage Result Mux and `ex_op` Encoding

4.5 Memory (MEM) Stage

4.5.1 Data Memory Organisation

4.6 Write-Back (WB) Stage

4.6.1 Writeback Mux

4.6.2 Cycle Counter and Instructions-Retired Counter

4.7 Pipeline Registers

4.8 Hazard Handling

4.8.1 Forwarding Unit

4.8.2 Hazard / Stall Unit

4.8.3 Branch Misprediction Flush

4.9 Branch Prediction Subsystem

4.9.1 BTB Organisation (16-entry, direct-mapped)

4.9.2 2-bit Saturating Counter FSM

- Phase 0 (single-cycle): all 47 RV32I instructions, register and memory state checked. **PASS.**
- Phase 1 (5-stage pipeline): identical expected state at termination, 120-cycle wait. **PASS.**
- Phase 2 (+BTB): same checks; BTB hit/miss observed. **PASS.**
- Phase 3 (+M-ext, custom-0): hand-assembled 13-instruction MAC microbenchmark. **PASS.**
- Phase 4 (+RDCYC, 3-op MAC): all nine benchmarks. **PASS.**
- Phase 6 (validation harness): 27-run sweep (9 benchmarks $\times$ 3 configs). **27/27 PASS.**

Chapter 5

Toolchain and Test Programs

5.1 RISC-V GCC Cross-Compilation Flow

5.2 Linker Script and Startup

5.3 Custom-Op Inline Assembly Wrappers

5.4 Build Pipeline (`tools/build.bat`)

5.5 Hand-Assembled Test Program

5.5.1 Assembly Listing

5.5.2 Manual Encoding to Machine Code

5.5.3 Instruction Memory and Data Memory Loading

5.6 Benchmark Suite

5.6.1 fib_20

5.6.2 dotprod_16 (custom and no-custom)

5.6.3 matmul_8x8 (custom and no-custom)

5.6.4 relu_32 (custom and no-custom)

5.6.5 grad_descent (custom and no-custom)

Chapter 6

Verification and Validation

6.1 Testbench Architecture

6.2 Validation Block Convention

6.3 Expected-Results Table

6.4 Instructions-Retired Counter and CPI

6.5 27-Run Automated Sweep

6.6 Functional Correctness Results

6.7 Register-File Dump (End of Execution)

6.8 Simulation Waveforms

Chapter 7

FPGA Synthesis and Timing Analysis

7.1 Target Platform

7.2 Code Changes for Synthesis

7.2.1 Memory Restructuring (4 Byte-Banks)

7.2.2 Output Ports to Prevent Logic Trimming

7.2.3 Clock Constraint (`constraints.xdc`)

7.3 Synthesis Flow

7.4 Resource Utilisation

7.5 Timing Results

7.5.1 Single-Cycle WNS and Critical Path

7.5.2 Pipeline WNS and Critical Path

7.5.3 Why the SC Max-Frequency Figure is Misleading

7.6 Real Execution Time Estimates

7.7 Pipeline Negation by Single-Cycle Multiplier

Chapter 8

Results and Analysis

8.1 Functional Correctness

8.2 Cycle Counts and CPI Across Configurations

8.3 BTB Speedup

8.4 Custom-Instruction Speedup

8.4.1 MAC Speedup on dotprod, matmul, grad_descent

8.4.2 The relu_32 Anomaly

8.5 Real-Time Performance from Synthesis

8.6 Pipeline-Negation Root Cause

8.7 Summary of Findings

- All 27 (benchmark, config) runs functionally correct.
- MAC delivers up to 3.45× speedup over software multiply.
- BTB delivers up to 1.33× speedup on branch-heavy code.
- Pipeline real-time is bottlenecked by the single-cycle multiplier; SC and pipeline have approximately equal true clock periods.

Chapter 9

Conclusion and Future Work

9.1 Conclusions

9.2 Limitations

9.3 Future Work

9.3.1 Multi-Stage Pipelined Multiplier

9.3.2 Caches and Larger Memories

9.3.3 Vector / SIMD Extension for ML

9.3.4 Hardware-Accelerated ReLU as Inline Fused Op

Bibliography

Appendix A

Complete Instruction Encoding Reference

Appendix B

Verilog File Manifest

The complete Verilog source listing is bound as a separate volume per the assignment requirement. This appendix indexes each module by phase.

Phase 0 — Single-Cycle Core

Phase 1 — Pipeline

Phase 2 — Branch Prediction

Phase 3 — M-Extension and Custom Ops

Phase 4 — RDCYC, 3-Operand MAC, Single-Cycle Top

Testbenches

Appendix C

Build and Simulation Instructions

C.1 Toolchain Setup

C.2 Building a C Benchmark

C.3 Running a Single Simulation in Vivado GUI

C.4 Headless 27-Run Sweep

C.5 Synthesis Flow